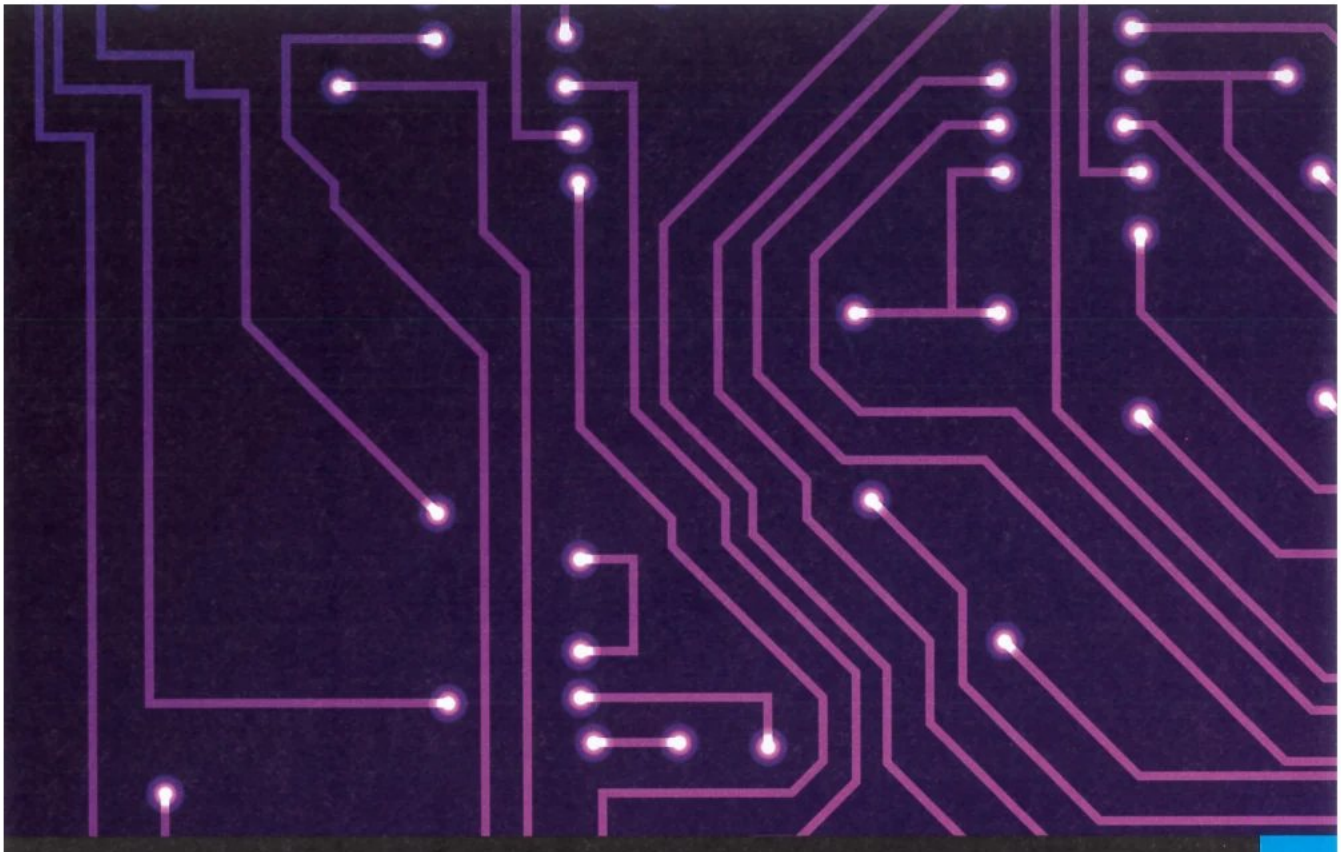


Chapter 23: Algorithms

Learning objectives

By the end of this chapter you should be able to:

- show understanding of linear and binary searching methods
- write a linear search algorithm
- write a binary search algorithm
- show understanding of the conditions necessary for the use of a binary search
- show understanding of how the performance of a binary search varies according to the number of data items
- show understanding of insertion sort and bubble sort methods
- show understanding that performance of a sorting routine may depend on the initial order of the data and the number of data items
- show understanding of and use Abstract Data Types (ADT)
- show how it is possible for ADTs to be implemented from another ADT
- describe the following ADTs and demonstrate how they can be implemented from appropriate built-in types or other ADTs: stack, queue, linked list, dictionary, binary tree
- write algorithms to:
  - implement an insertion sort
  - implement a bubble sort
  - find an item in each of the following: linked list, binary tree
  - insert an item into each of the following: stack, queue, linked list, binary tree
  - delete an item from each of the following: stack, queue, linked list
- show understanding that a graph is an example of an ADT
- describe the key features of a graph and justify its use for a given situation
- show understanding that different algorithms which perform the same task can be compared by using criteria such as time taken to complete the task and memory used
- show understanding of Big O notation for time and space complexity.

23.01 Linear search

In Chapter 13, we developed the algorithm for a linear search (Worked Example 13.02).

Discussion Point:

What were the essential features of a linear search?

TASK 23.01

Write program code for the linear search algorithm. Assume that the items to be searched are stored in a 1D array with n elements.

23.02 Bubble sort

In Chapter 13, we developed the algorithm for a bubble sort (Worked Example 13.03).

Discussion Point:

What were the essential features of a bubble sort?

TASK 23.02

Write program code for the most efficient bubble sort algorithm. Assume that the items to be sorted are stored in a 1D array with n elements.

23.03 Insertion sort

Imagine you have a number of cards with a different value printed on each card. How would you sort these cards into order of increasing value?

You can consider the pile of cards as consisting of a sorted part and an unsorted part. Place the unsorted cards in a pile on the table. Hold the sorted cards as a pack in your hand. To start with only the first (top) card is sorted. The card on the top of the pile on the table is the next card to be inserted. The last (bottom) card in your hand is your current card.

Figure 23.01 shows the sorted cards in your hand as blue and the pile of unsorted cards as white. The next card to be inserted is shown in red. Each column shows the state of the pile as the cards are sorted.

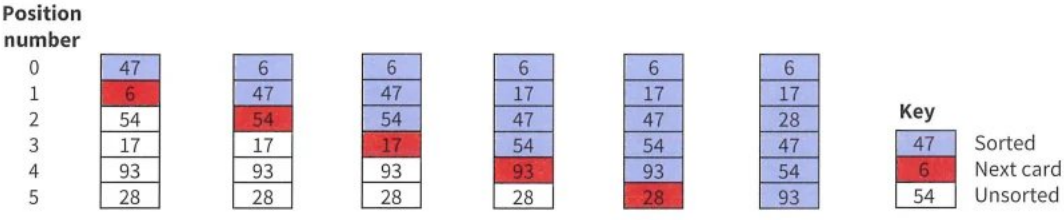


Figure 23.01 Sorting cards

Repeat the following steps until all cards in the unsorted pile have been inserted into the correct position.

1 Repeat until the card to be inserted has been placed in its correct position.

1.1 Compare the current card with the card to be inserted.

1.2 If the card to be inserted is greater than the current card, insert it below the current card.

1.3 Otherwise, if there is a card above the current card in your hand, make this your new current card.

1.4 If there is no new current card, place the card to be inserted at the top of the sorted pile.

What happens when you work through the sorted cards to find the correct position for the card to be inserted? In effect, as you consider the cards in your hand, you move the current card down a position. If the value of the card to be inserted is smaller than the last card you considered, then the card is inserted at the top of the pile (position 1).

This method is known as an insertion sort. It is a standard sort method.

We can write this algorithm using pseudocode. Assume the values to be sorted are stored in a 1D array, `List`:

```
FOR Pointer ← 1 TO NumberOfitems - 1
    ItemToBeInserted ← List[Pointer]
    CurrentItem ← Pointer - 1 // pointer to last item in sorted part of list
    WHILE (List[CurrentItem] > ItemToBeInserted) AND (CurrentItem > -1) DO
        List[CurrentItem + 1] ← List[CurrentItem] // move current item down
        CurrentItem ← CurrentItem - 1 // look at the item above
    ENDWHILE
    List[CurrentItem + 1] ← ItemToBeInserted // insert item
NEXT Pointer
```

TASK 23.03

1 Dry-run the insertion sort algorithm using a trace table. Assume the list consists of the following six items in the order given: 53, 21, 60, 18, 42, 19.

2 Write program code for the insertion sort algorithm. Assume that the items to be sorted are stored in a 1D array with n elements.

Extension Question 23.01

Investigate the performances of the insertion sort and the bubble sort by:

- varying the initial order of the items
- increasing the number of items to be sorted.

23.04 Binary search

In Section 23.01 we considered the algorithm for a linear search. This is the only way we can systematically search an unordered list. However, if the list is ordered, then we can use a different technique.

Consider the following real-world example.

If you want to look up a word in a dictionary, you are unlikely to start searching for the word from the beginning of the dictionary. Suppose you are looking for the word 'quicksort'. You look at the middle entry of the dictionary (approximately) and find the word 'magnetic'. 'quicksort' comes after 'magnetic', so you look in the second half of the dictionary. Again, you look at the entry in the middle of this second half of the dictionary (approximately) and find the word 'report'. 'quicksort' comes before 'report', so you look in the third quarter. You can keep looking at the middle entry of the part which must contain your word, until you find the word. If the word does not exist in the dictionary, you will have no entries in the dictionary left to find the middle of.

This method is known as a **binary search**. It is a standard method.



KEY TERM

Binary search: repeated checking of the middle item in an ordered search list and discarding the half of the list which does not contain the search item

We can write this algorithm using pseudocode. Assume the values are sorted in ascending order and stored in a 1D array, `List` of size `MaxItems`.

```
Found ← FALSE
SearchFailed ← FALSE
First ← 0
Last ← MaxItems - 1 // set boundaries of search area
WHILE NOT Found AND NOT SearchFailed DO
    Middle ← (First + Last) DIV 2 // find middle of current search area
    IF List[Middle] = SearchItem
      THEN
        Found ← TRUE
      ELSE
        IF First >= Last // no search area left
          THEN
            SearchFailed ← TRUE
          ELSE
            IF List[Middle] > SearchItem
              THEN  // must be in first half
                Last ← Middle - 1  // move upper boundary
              ELSE  // must be in second half
                First ← Middle + 1  // move lower boundary
            ENDIF
        ENDIF
    ENDIF
ENDWHILE
IF Found = TRUE
  THEN
    OUTPUT Middle // output position where item was found
  ELSE
    OUTPUT "Item not present in array"
ENDIF
```

TASK 23.04

1. Dry-run the binary search algorithm using a trace table. Assume the list consists of the following 20 items in the order given: 7, 12, 19, 23, 27, 33, 37, 41, 45, 56, 59, 60, 62, 71, 75, 80, 84, 88, 92, 99.
2. Search for the value 60. How many times did you have to execute the `While` loop?
3. Dry-run the algorithm again, this time searching for the value 34. How many times did you have to execute the While loop?

Discussion Point:

Compare the binary-search algorithm with the linear-search algorithm. If the array contains n items, how many times on average do you need to test a value when using a binary search and how many times on average do you need to test a value when using a linear search? Can you describe how the search time varies with increasing n?

23.05 Abstract Data Types (ADTs)

In Chapter 13 (Section 13.07 to 13.10), we introduced ADTs using conceptual diagrams and how ADTs can be implemented using arrays.

23.06 Linked lists

Look back at Chapter 13 Section 13.10. Figure 13.25 shows an empty linked list. The `StartPointer` variable contains the Null pointer. The free list links all empty nodes.

We now code the basic operations discussed using the conceptual diagrams in Figures 13.11 to 13.16.

Create a new linked list

```
// NullPointer should be set to -1 if using array element with index 0
CONSTANT NullPointer = -1
// Declare record type to store data and pointer
TYPE ListNode
    DECLARE Data    : STRING
    DECLARE Pointer : INTEGER
ENDTYPE
DECLARE StartPointer : INTEGER
DECLARE FreeListPtr  : INTEGER
DECLARE List : ARRAY[0 : 6] OF ListNode

PROCEDURE InitialiseList
    StartPointer ← NullPointer       // set start pointer
    FreeListPtr ← 0                  // set starting position of free list
    FOR Index ← 0 TO 5               // link all nodes to make free list
        List[Index].Pointer ← Index + 1
    NEXT Index
    List[6].Pointer ← NullPointer    // last node of free list
ENDPROCEDURE
```

Insert a new node into an ordered linked list

```
PROCEDURE InsertNode(NewItem)
    IF FreeListPtr <> NullPointer
      THEN  // there is space in the array
        // take node from free list and store data item
        NewNodePtr ← FreeListPtr
        List[NewNodePtr].Data ← NewItem
        FreeListPtr ← List[FreeListPtr].Pointer
        // find insertion point
        ThisNodePtr ← StartPointer  // start at beginning of list
        PreviousNodePtr ← NullPointer
        WHILE ThisNodePtr <> NullPointer  // while not end of list
          AND List[ThisNodePtr].Data < NewItem DO
            PreviousNodePtr ← ThisNodePtr  // remember this node
            // follow the pointer to the next node
            ThisNodePtr ← List[ThisNodePtr].Pointer
        ENDWHILE
        IF PreviousNodePtr = StartPointer
          THEN  // insert new node at start of list
            List[NewNodePtr].Pointer ← StartPointer
            StartPointer ← NewNodePtr
          ELSE  // insert new node between previous node and this node
            List[NewNodePtr].Pointer ← List[PreviousNodePtr].Pointer
            List[PreviousNodePtr].Pointer ← NewNodePtr
        ENDIF
    ENDIF
ENDPROCEDURE
```


After three data items have been added to the linked list, the array contents are as shown in Figure 23.02.

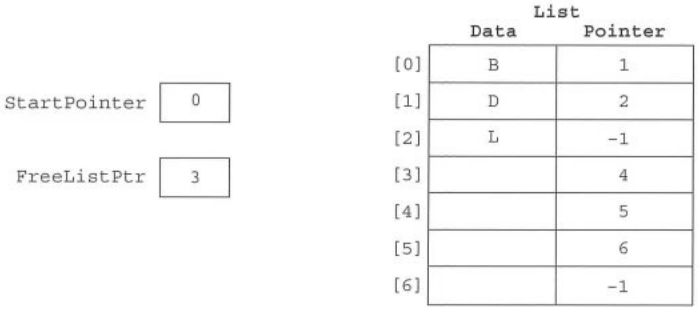


Figure 23.02 Linked list of three nodes and free list of four nodes

Find an element in an ordered linked list

```
FUNCTION FindNode(DataItem) RETURNS INTEGER  // returns pointer to node
    CurrentNodePtr ← StartPointer  // start at beginning of list
    WHILE CurrentNodePtr <> NullPointer  // not end of list
      AND List[CurrentNodePtr].Data <> DataItem DO // item not found
        // follow the pointer to the next node
        CurrentNodePtr ← List[CurrentNodePtr].Pointer
```

```
    ENDWHILE
    RETURN CurrentNodePtr  // returns NullPointer if item not found
ENDFUNCTION
```

Delete a node from an ordered linked list

```
PROCEDURE DeleteNode(DataItem)
    ThisNodePtr ← StartPointer              // start at beginning of list
    WHILE ThisNodePtr <> NullPointer        // while not end of list
      AND List[ThisNodePtr].Data <> DataItem DO // and item not found
        PreviousNodePtr ← ThisNodePtr  // remember this node
        // follow the pointer to the next node
        ThisNodePtr ← List[ThisNodePtr].Pointer
    ENDWHILE
    IF ThisNodePtr <> NullPointer  // node exists in list
      THEN
        IF ThisNodePtr = StartPointer // first node to be deleted
          THEN
            // move start pointer to the next node in list
            StartPointer ← List[StartPointer].Pointer
          ELSE
            // it is not the start node;
            // so make the previous node's pointer point to
            // the current node's 'next' pointer; thereby removing all
            // references to the current pointer from the list
            List[PreviousNodePtr].Pointer ← List[ThisNodePtr].Pointer
        ENDIF
        List[ThisNodePtr].Pointer ← FreeListPtr
        FreeListPtr ← ThisNodePtr
    ENDIF
ENDPROCEDURE
```

Access all nodes stored in the linked list

```
PROCEDURE OutputAllNodes
    CurrentNodePtr ← StartPointer // start at beginning of list
    WHILE CurrentNodePtr <> NullPointer DO // while not end of list
        OUTPUT List[CurrentNodePtr].Data
        // follow the pointer to the next node
        CurrentNodePtr ← List[CurrentNodePtr].Pointer
    ENDWHILE
ENDPROCEDURE
```

TASK 23.05

Convert the pseudocode for the linked-list handling subroutines to program code. Incorporate the subroutines into a program and test them.

Note that a stack ADT and a queue ADT can be treated as special cases of linked lists. The linked list stack only needs to add and remove nodes from the front of the linked list (see Section 23.08). The linked list queue only needs to add nodes to the end of the linked list and remove nodes from the front of the linked list (see Section 23.09).

23.07 Binary trees

In the real world, we draw tree structures to represent hierarchies. For example, we can draw a family tree showing ancestors and their children. A binary tree is different to a family tree because each node can have at most two 'children'.

In computer science binary trees are used for different purposes. In Chapter 20 (Section 20.06), you saw the use of a binary tree as a syntax tree. In this chapter, you will use an ordered binary tree ADT (such as the one shown in Figure 23.03) as a binary search tree.

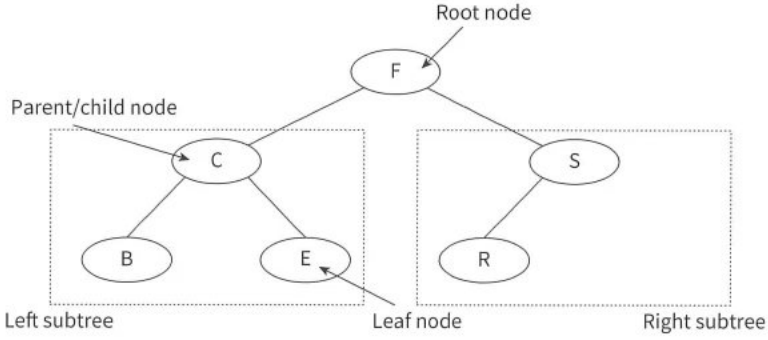


Figure 23.03 Conceptual diagram of an ordered binary tree

Nodes are added to an ordered binary tree in a specific way:

Start at the root node as the current node.

Repeat

If the data value is greater than the current node's data value, follow the right branch.

If the data value is smaller than the current node's data value, follow the left branch.

Until the current node has no branch to follow.

Add the new node in this position.

For example, if we want to add a new node with data value D to the binary tree in Figure 23.03, we execute the following steps.

1. Start at the root node.
2. D is smaller than F, so turn left.
3. D is greater than C, so turn right.
4. D is smaller than E, so turn left.
5. There is no branch going left from E, so we add D as a left child from E (see Figure 23.04).

This type of tree has a special use as a search tree. Just like the binary search applied to an ordered linear list, the binary search tree gives the benefit of a faster search than a linear search or searching a linked list. The ordered binary tree also has a benefit when adding a new node: other nodes do not need to be moved, only a left or right pointer needs to be added to link the new node into the existing tree.

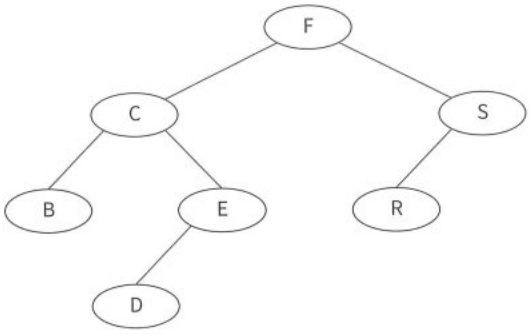


Figure 23.04 Conceptual diagram of adding a node to an ordered binary tree

We can store the binary tree in an array of records (see Figure 23.05). One record represents a node and consists of the data and a left pointer and a right pointer. Unused nodes are linked together to form a free list.

RootPointer: -1

FreePtr: 0

Tree

	LeftPointer	Data	RightPointer
[0]	1		
[1]	2		
[2]	3		
[3]	4		
[4]	5		
[5]	6		
[6]	-1		

Figure 23.05 Binary tree before any nodes are inserted

Create a new binary tree

```
// NullPointer should be set to -1 if using array element with index 0
CONSTANT NullPointer = -1
// Declare record type to store data and pointers
TYPE TreeNode
    DECLARE Data : STRING
    DECLARE LeftPointer : INTEGER
    DECLARE RightPointer : INTEGER
ENDTYPE
DECLARE RootPointer : INTEGER
DECLARE FreePtr : INTEGER
DECLARE Tree : ARRAY[0 : 6] OF TreeNode
PROCEDURE InitialiseTree
    RootPointer ← NullPointer  // set start pointer
    FreePtr ← 0                // set starting position of free list
    FOR Index ← 0 TO 5         // link all nodes to make free list
        Tree[Index].LeftPointer ← Index + 1
    NEXT Index
    Tree[6].LeftPointer ← NullPointer // last node of free list
ENDPROCEDURE
```

Insert a new node into a binary tree

```
PROCEDURE InsertNode(NewItem)
    IF FreePtr <> NullPointer
      THEN  // there is space in the array
        // take node from free list, store data item, set null pointers
        NewNodePtr ← FreePtr
        FreePtr ← Tree[FreePtr].LeftPointer
        Tree[NewNodePtr].Data ← NewItem
        Tree[NewNodePtr].LeftPointer ← NullPointer
        Tree[NewNodePtr].RightPointer ← NullPointer
        // check if empty tree
        IF RootPointer = NullPointer
          THEN  // insert new node at root
            RootPointer ← NewNodePtr
          ELSE  // find insertion point
            ThisNodePtr ← RootPointer   // start at the root of the tree
            WHILE ThisNodePtr <> NullPointer DO  // while not a leaf node
                PreviousNodePtr ← ThisNodePtr  // remember this node
                IF Tree[ThisNodePtr].Data > NewItem
                  THEN  // follow left pointer
                    TurnedLeft ← TRUE
                    ThisNodePtr ← Tree[ThisNodePtr].LeftPointer
                  ELSE // follow right pointer
                    TurnedLeft ← FALSE
                    ThisNodePtr ← Tree[ThisNodePtr].RightPointer
                ENDIF
            ENDWHILE
            IF TurnedLeft = TRUE
              THEN
                Tree[PreviousNodePtr].LeftPointer ← NewNodePtr
              ELSE
                Tree[PreviousNodePtr].RightPointer ← NewNodePtr
            ENDIF
        ENDIF
    ENDIF
ENDPROCEDURE
```

Find a node in a binary tree

```
FUNCTION FindNode(SearchItem) RETURNS INTEGER  // returns pointer to node
    ThisNodePtr ← RootPointer      // start at the root of the tree
    WHILE ThisNodePtr <> NullPointer   // while a pointer to follow
      AND Tree[ThisNodePtr].Data <> SearchItem DO // and search item not found
        IF Tree[ThisNodePtr].Data > SearchItem
          THEN  // follow left pointer
            ThisNodePtr ← Tree[ThisNodePtr].LeftPointer
          ELSE // follow right pointer
            ThisNodePtr ← Tree[ThisNodePtr].RightPointer
        ENDIF
    ENDWHILE
```

```
    RETURN ThisNodePtr // will return null pointer if search item not found
ENDFUNCTION
```

TASK 23.06

Write program code to implement a binary search tree.

23.08 Stacks

In Chapter 13 (Section 13.08) we looked at the conceptual data structure of a stack. A stack can be implemented using a 1D array. Figure 23.06 shows a stack containing four data items. For conceptual reasons the array elements are numbered from the bottom up.

Note that `BaseOfStackPointer` will always point to element 0 of the array. `TopOfStackPointer` will vary. It will increase when an item is pushed onto the stack and it will decrease when an item is popped off the stack. When the stack is empty, `TopOfStackPointer` will have the value -1.

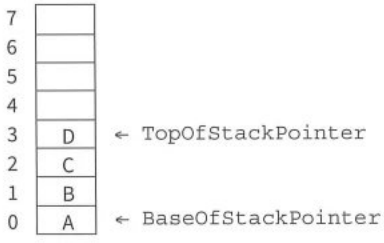


Figure 23.06 A stack

Create a new stack

```
// NullPointer should be set to -1 if using array element with index 0
CONSTANT EMPTYSTRING = ""
CONSTANT NullPointer = -1
CONSTANT MaxStackSize = 8
DECLARE BaseOfStackPointer : INTEGER
DECLARE TopOfStackPointer : INTEGER
DECLARE Stack : ARRAY[1 : MaxStackSize - 1] OF STRING

PROCEDURE InitialiseStack
    BaseOfStackPointer ← 0              // set base of stack pointer
    TopOfStackPointer ← NullPointer     // set top of stack pointer
ENDPROCEDURE
```

Push an item onto the stack

```
PROCEDURE Push(NewItem)
    IF TopOfStackPointer < MaxStackSize - 1
      THEN      // there is space on the stack
                // increment top of stack pointer
        TopOfStackPointer ← TopOfStackPointer + 1
                // add item to top of stack
        Stack[TopOfStackPointer] ← NewItem
    ENDIF
ENDPROCEDURE
```

Pop an item off the stack

```
FUNCTION Pop()
    DECLARE Item : STRING
    Item ← EMPTYSTRING
    IF TopOfStackPointer > NullPointer
      THEN      // there is at least one item on the stack
                // pop item off the top of the stack
        Item ← Stack[TopOfStackPointer]
                // decrement top of stack pointer
        TopOfStackPointer ← TopOfStackPointer - 1
    ENDIF
    RETURN Item
ENDFUNCTION
```

TASK 23.07

1 Write a program to implement the above pseudocode subroutines. Add a menu to test the subroutines.

2 Write a program to implement a stack as a linked list. Note that the adding and removing of nodes is much simpler than for an ordered linked list.

23.09 Queues

In Chapter 13 (Section 13.09) we looked at the conceptual data structure of a queue. A queue can be implemented using a 1D array. Figure 23.07 shows a queue containing five data items.

`FrontOfQueuePointer` always points to the first element in the queue, that is the next element to be taken from the queue. `EndOfQueuePointer` always points to the last element in the queue. Before another element joins the queue, the `EndOfQueuePointer` is incremented. Note that when adjusting either pointer the possibility of wrap-round has to be tested.

To make it easier to test whether the queue is empty or full, a counter variable can be used.

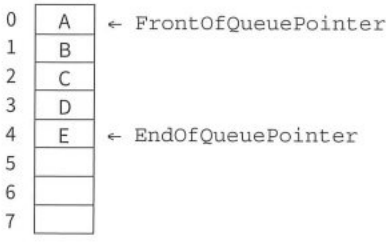


Figure 23.07 A queue before wrap-round

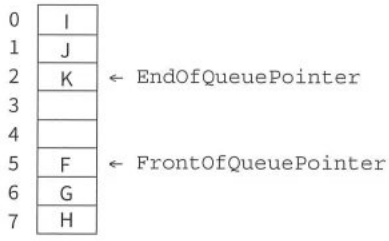


Figure 23.08 A queue with wrap-round

Create a new queue

```
// NullPointer should be set to -1 if using array element with index 0
CONSTANT EMPTYSTRING = ""
CONSTANT NullPointer = -1
CONSTANT MaxQueueSize = 8
DECLARE FrontOfQueuePointer : INTEGER
DECLARE EndOfQueuePointer : INTEGER
DECLARE NumberInQueue : INTEGER
DECLARE Queue : ARRAY[0 : MaxQueueSize - 1] OF STRING
PROCEDURE InitialiseQueue
    FrontOfQueuePointer ← NullPointer    // set front of queue pointer
```

```
    EndOfQueuePointer ← NullPointer      // set end of queue pointer
    NumberInQueue ← 0                    // no elements in queue
ENDPROCEDURE
```

Add an item to the queue

```
PROCEDURE AddToQueue(NewItem)
    IF NumberInQueue < MaxQueueSize
      THEN      // there is space in the queue
                // increment end of queue pointer
        EndOfQueuePointer ← EndOfQueuePointer + 1
                // check for wrap-round
        IF EndOfQueuePointer > MaxQueueSize - 1
          THEN      // wrap to beginning of array
            EndOfQueuePointer ← 0
                    // add item to end of queue
        ENDIF
        Queue[EndOfQueuePointer] ← NewItem
         // increment counter
         NumberInQueue ← NumberInQueue + 1
    ENDIF
ENDPROCEDURE
```

Remove an item from the queue

```
FUNCTION RemoveFromQueue()
    DECLARE Item : STRING
    Item ← EMPTYSTRING
    IF NumberInQueue > 0
      THEN // there is at least one item in the queue
           // remove item from the front of the queue
        Item ← Queue[FrontOfQueuePointer]
        NumberInQueue ← NumberInQueue - 1
        IF NumberInQueue = 0
          THEN // if queue empty, reset pointers
            CALL InitialiseQueue
          ELSE
            // increment front of queue pointer
            FrontOfQueuePointer ← FrontOfQueuePointer + 1
            // check for wrap-round
            IF FrontOfQueuePointer > MaxQueueSize - 1
              THEN // wrap to beginning of array
              FrontOfQueuePointer ← 0
            ENDIF
        ENDIF
    ENDIF
    RETURN Item
ENDFUNCTION
```

TASK 23.08

1 Write a program to implement the above pseudocode subroutines. Add a menu to test the subroutines.

2 Write program code to implement a queue as a linked list. You may find it helpful to introduce another pointer that always points to the end of the queue. You will need to update it when you add a new node to the queue.

23.10 Graphs

In Computer Science a graph is an ADT consisting of vertices (nodes) and edges. Graphs are used to record relationships between things. For uses of graphs in AI see Chapter 22 Section 22.02. A simple graph is shown in Figure 23.09. It represents a small part of the London Underground map shown in Figure 12.02 in Chapter 12.

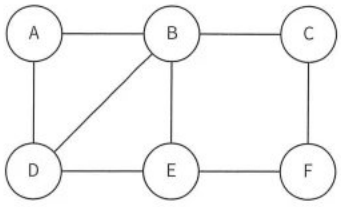


Figure 23.09 Graph showing part of the London Underground map

The vertices labelled A to F are the underground stations and the edges represent train lines connecting the stations. For example, you can take a train directly from B to D. To get from B to F, you have to travel via C or E. Two vertices connected by an edge are known as neighbours.

A labelled (weighted) graph has edges with values representing something. In our example, we can add weights to Figure 23.10 to show the time it takes to travel between stations:

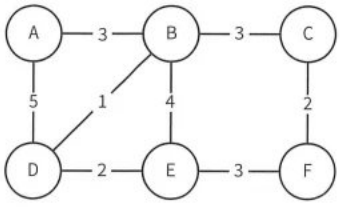


Figure 23.10 Weighted graph showing travelling times between stations

Graphs can be directed or undirected (as in Figure 23.10). Travel times may vary depending on the direction of travel. This can be shown in a directed graph (see Figure 23.11).

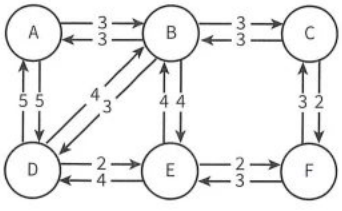


Figure 23.11 Directed graph

Sometimes one direction may not be available. For example, if the line from B to D is blocked, this could be represented as shown in Figure 23.12.

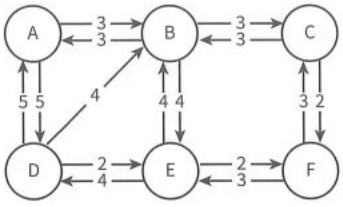


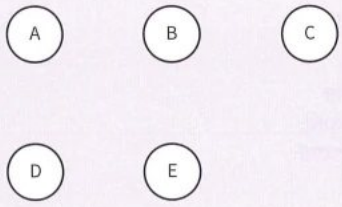
Figure 23.12 Directed graph

We can use a graph to plan a journey. Using Figure 23.12, we want to travel from C to D, so we can either use the route C B A D (11 minutes) or the route C F E D (9 minutes). For the return journey we can use D B C (7 minutes).

TASK 23.09

Draw the labelled edges for a directed graph of a road system where:

- A to B is 2 km, and it is a one-way system
- D to A is 1 km, and is one-way
- B to D is 3 km and is one-way
- B to C is 4 km
- C to E is 7 km
- D to E is 5 km.



To implement a graph, we can use an adjacency matrix or an adjacency list.

An adjacency matrix stores the relationship between every vertex to all other vertices. For an unweighted graph, a 1 represents an edge, a 0 no edge. When weights are to be recorded, the weight replaces the 1. Instead of a 0, we use the infinity symbol ∞.

	A	B	C	D	E	F
A	0	1	0	1	0	0
B	1	0	1	1	1	0
C	0	1	0	0	0	1
D	1	1	0	0	1	0
E	0	1	0	1	0	1
F	0	0	1	0	1	0

Table 23.01

Adjacency matrix for Figure 23.09

	A	B	C	D	E	F
A	∞	3	∞	5	∞	∞
B	3	∞	3	∞	4	∞
C	∞	3	∞	∞	∞	2
D	5	4	∞	∞	2	∞
E	∞	4	∞	4	∞	2
F	∞	∞	3	∞	3	∞

Table 23.02

Adjacency matrix for Figure 23.12

An adjacency list stores the relationship between every vertex to all relevant vertices. An entry is made only when there is an edge between two vertices. For a weighted graph, the connection as well as the weight is stored in the list.

	connected to
A	B, D
B	A, C, D, E
C	B, F
D	A, B, E
E	B, D, F
F	C, E

Table 23.03

Adjacency list for Figure 23.09

	connected to
A	B, 3; D, 5
B	A, 3; C, 3; E, 4
C	B, 3; F, 2
D	A, 5; B, 4; E, 2
E	B, 4; D, 4; F, 2
F	C, 3; E, 3

Table 23.04

Adjacency list for Figure 23.12

TASK 23.10

Construct an adjacency matrix and an adjacency list to represent your graph from Task 23.09.

23.11 Hash tables

If we want to store records in an array and have direct access to records, we can use the concept of a hash table.

The idea behind a hash table is that we calculate an address (the array index) from the key value of the record and store the record at this address. When we search for a record, we calculate the address from the key and go to the calculated address to find the record. Calculating an address from a key is called 'hashing'.

Finding a hashing function that will give a unique address from a unique key value is very difficult. If two different key values hash to the same address this is called a 'collision'. There are different ways to handle collisions:

- chaining: create a linked list for collisions with start pointer at the hashed address
- using overflow areas: all collisions are stored in a separate overflow area, known as 'closed hashing'
- using neighbouring slots: perform a linear search from the hashed address to find an empty slot, known as 'open hashing'.

WORKED EXAMPLE 23.01

Calculating addresses in a hash table

Assume we want to store customer records in a 1D array `HashTable[0 : n]`. Each customer has a unique customer ID, an integer in the range 10001 to 99999.

We need to design a suitable hashing function. The result of the hashing function should be such that every index of the array can be addressed directly. The simplest hashing function gives us addresses between 0 and n:

(Continued)

```
FUNCTION Hash(Key) RETURNS INTEGER
    Address ← Key MOD(n + 1)
    RETURN Address
ENDFUNCTION
```

For illustrative purposes, we choose n to be 9. Our hashing function is:

```
Index ← CustomerID MOD 10
```

We want to store records with customer IDs: 45876, 32390, 95312, 64636, 23467. We can store the first three records in their correct slots, as shown in Figure 23.13.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
32390		95312				45876			

Figure 23.13 A hash table without collisions

The fourth record key (64636) also hashes to index 6. This slot is already taken; we have a collision. If we store our record here, we lose the previous record. To resolve the collision, we can choose to store our record in the next available space, as shown in Figure 23.14.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
32390		95312				45876	64636		

Figure 23.14 A hash table with a collision resolved by open hashing

The fifth record key (23467) hashes to index 7. This slot has been taken up by the previous record, so again we need to use the next available space (Figure 23.15).

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
32390		95312				45876	64636	23467	

Figure 23.15 A hash table with two collisions resolved by open hashing

When searching for a record, we need to allow for these out-of-place records. We know if the record we are searching for does not exist in the hash table when we come across an unoccupied slot.

We will now develop algorithms to insert a record into a hash table and to search for a record in the hash table using its record key.

The hash table is a 1D array `HashTable[0 : Max] OF Record`.

The records stored in the hash table have a unique key stored in field `Key`.

Insert a record into a hash table

```
PROCEDURE Insert(NewRecord)
    Index ← Hash(NewRecord.Key)
    WHILE HashTable[Index] NOT empty DO
        Index ← Index + 1 // go to next slot to check if empty
        IF Index > Max // beyond table boundary?
          THEN // wrap around to beginning of table
            Index ← 0
        ENDIF
    ENDWHILE
```

```
    HashTable[Index] ← NewRecord
ENDPROCEDURE
```

Find a record in a hash table

```
FUNCTION FindRecord(SearchKey) RETURNS Record
    Index ← Hash(SearchKey)
    WHILE (HashTable[Index].Key <> SearchKey)
      AND (HashTable[Index] NOT empty) DO
        Index ← Index + 1 // go to next slot
        IF Index > Max // beyond table boundary?
          THEN // wrap around to beginning of table
            Index ← 0
        ENDIF
    ENDWHILE
    IF HashTable[Index] NOT empty // if record found
      THEN
        RETURN HashTable[Index] // return the record
    ENDIF
ENDFUNCTION
```

23.12 Dictionaries

A real-world dictionary is a collection of key–value pairs. The key is the term you use to look up the required value. For example, if you use an English–French dictionary to look up the English word 'book', you will find the French equivalent word 'livre'. A real-world dictionary is organised in alphabetical order of keys.

An ADT dictionary in computer science is implemented using a hash table (see Section 23.11), so that a value can be looked up using a direct-access method.

Python, VB.NET and Java have a built-in ADT dictionary class.

Here are some examples of Python dictionaries:

```
EnglishFrench = {} # empty dictionary
EnglishFrench["book"] = "livre" # add a key-value pair to the dictionary
EnglishFrench["pen"] = "stylo"
print(EnglishFrench["book"]) # access a value in the dictionary
# alternative method of setting up a dictionary
ComputingTerms = {"Boolean" : "can be TRUE or FALSE", "Bit" : "0 or 1"}
print(ComputingTerms["Bit"])
```

Here are some examples of VB dictionaries:

```
Dim EnglishFrench As New Dictionary(Of String, String)
EnglishFrench.Add("book", "livre")
EnglishFrench.Add("pen", "stylo")
Console.WriteLine(EnglishFrench.Item("book"))
Dim ComputingTerms As New Dictionary(Of String, String)
ComputingTerms.Add("Boolean", "can be TRUE or FALSE")
ComputingTerms.Add("Bit", "0 or 1")
Console.WriteLine(ComputingTerms.Item("Bit"))
Console.ReadLine()
```

Here are some examples of Java dictionaries (the Dictionary class is obsolete, use HashMap instead):

```
import java.util.Map;
import java.util.HashMap;
    Map<String, String> englishFrench = new HashMap<String, String>();
    englishFrench.put("book", "livre");
    englishFrench.put("pen", "stylo");
    System.out.println(englishFrench.get("book"));
    Map<String, String>  computingTerms = new HashMap<String, String>();
    computingTerms.put("Boolean", "can be TRUE or FALSE");
    computingTerms.put("Bit", "0 or 1");
    System.out.println(computingTerms.get("Bit"));
```

There are many built-in functions for Python, VB and Java dictionaries. These are beyond the scope of this book. However, we need to understand how dictionaries are implemented. The following pseudocode shows how to create a new dictionary.

```
TYPE DictionaryEntry
    DECLARE Key   : STRING
    DECLARE Value : STRING
ENDTYPE
DECLARE EnglishFrench : ARRAY[0 : 999] OF DictionaryEntry  // empty
dictionary
```

TASK 23.11

Write pseudocode to:

- insert a key–value pair into a dictionary
- look up a value in a dictionary.

Use the hashing function from Worked Example 23.01.

23.13 Big O notation

A problem can be solved in different ways, with different algorithms. Clearly, we want to use time and memory efficiently. A way of comparing the efficiency of algorithms has been devised using order of growth as a function of the size of the input. Big O notation is used to classify algorithms according to how their running time (or space requirements) grows as the input size grows. The letter O is used because the growth rate of a function is also referred to as 'order of the function'. The worst-case scenario is used when calculating the order of growth for very large data sets.

Consider the linear search algorithm in Chapter 13, Worked Example 13.02. The worst case scenario is that the item searched for is the last item in the list. The longer the list, the more comparisons have to be made. If the list is twice as long, twice as many comparisons have to be made. Generally, we can say the order of growth is linear. We write this as O(n), where n is the size of the data set.

Consider the bubble sort algorithm for the worst case scenario.

```
Unsorted ← n - 1
FOR i ← 0 TO n - 2
    FOR j ← 0 TO Unsorted - 1
        IF MyList[j] > MyList[j + 1]
```

```
            THEN
              Temp ← MyList[j]
              MyList[j] ← MyList[j + 1]
              MyList[j + 1] ← Temp
            ENDIF
        NEXT j
        Unsorted ← Unsorted - 1
    NEXT i
```

The basic operation for this algorithm is the comparison `IF MyList[j] > MyList[j + 1]`

n	Number of comparisons	
1	0	
2	1	
3	3	= 1 + 2
4	6	= 1 + 2 + 3
5	10	= 1 + 2 + 3 + 4
6	15	= 1 + 2 + 3 + 4 + 5

Table 23.05 Number of comparisons in bubble sort

We can see that the total number of comparisons is the sum of the first (n – 1) whole numbers. This leads us to the formula:

$½ * n * (n - 1) = ½ * (n^2 - n)$

For very large n we can disregard all factors except the largest and we get n^2. So the order of growth is n^2.

Consider the binary search algorithm in Section 23.04. With each iteration this algorithm halves the number of values in the data set. This iterative halving of data sets produces a growth curve that peaks at the beginning and slowly flattens out as the size of the data sets increase. This type of algorithm is described as $O(\log_2 n)$

Table 23.06 shows a summary of standard algorithms and their order of growth (time complexity).

Order of growth	Example	Explanation
O(1)	`FUNCTION GetFirstItem(List : ARRAY) RETURN List[1]`	The complexity of the algorithm does not change regardless of data set size
O(n)	Linear search Bubble sort performed on an already sorted list	Linear growth
$O(\log_2 n)$	Binary search	The total time taken increases as the data set size increases, but each comparison halves the data set. So the time taken increases by smaller amounts and approaches constant time.
$O(n^2)$	Bubble sort Insertion sort	Polynomial growth Common with algorithms that involve nested iterations over the data set
$O(n^3)$		Polynomial growth Deeper nested iterations will result in $O(n^3)$, $O(n^4)$, …
$O(2^n)$	Recursive calculation of Fibonacci numbers	Exponential growth

Table 23.06 Order of growth (time complexity) for data input set of size n

Space complexity refers to the amount of memory required for a growing number of values n in the dataset.

The ADT operations discussed in this chapter have space complexity O(n). This means that they only take the space required for the data set. Bubble sort and insertion sort have space complexity of O(1). This means they do not require extra memory for sorting larger lists.

Reflection Point:

List the standard algorithms you have met in this chapter. Can you give the essential features of each of these?

Summary

- Standard algorithms include bubble sort, insertion sort, linear search and binary search.
- Abstract data types (ADTs) include stacks, queues, linked lists, binary trees, hash tables, dictionaries and graphs.
- Basic operations required for an ADT include creating an ADT and inserting, finding or deleting an element of an ADT.
- Time taken and space used can be measured using Big O notation.

Exam-style Questions

1 a Complete the algorithm for a binary search function `FindName`.
The data being searched is stored in the array `Names[0 : 50]`.
The name to be searched for is passed as a parameter.

```
FUNCTION FindName(s : STRING) RETURNS INTEGER
    Index ← -1
    First ← 0
    Last ← 50
    WHILE (Last >= First) AND ..................................... DO
        Middle ← (First + Last) DIV 2
        IF Names[Middle] = s
          THEN
            Index ← Middle
          ELSE
            IF ..................................................
              THEN
                Last ← Middle + 1
              ELSE
                ..................................................
            ENDIF
        ENDIF
    ENDWHILE
ENDFUNCTION
```

[3]

b The binary search does not work if the data in the array being searched is ..... [1]

c What does the function `FindName` return when:

i the name searched for exists in the array?

ii the name searched for does not exist in the array? [2]